Slurm and CUDA

Quick Slurm Guide

Why Use a Cluster?

- Many users share compute resources
- Jobs are managed by Slurm
- GPUs must be requested explicitly
- Programs usually should not run on login nodes

Typical Workflow

```
ssh user@cluster
module load cuda
nvcc main.cu -o main
sbatch job.sh
squeue -u $USER
```

Module System

- $ module avail / load x / list / unload x

- Loads compilers, CUDA, and libraries
- Available versions depend on the cluster

sbatch

- $ sbatch job.sh

- Submits a batch job
- Job runs later through the scheduler
- Output is often written to slurm-<jobid>.out

Simple sbatch Script

```
#!/bin/bash
#SBATCH --job-name=cuda_test
#SBATCH --output=out.txt
#SBATCH --time=00:10:00
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=4
#SBATCH --mem=4G

module load cuda
./main
```

squeue and scancel

- $ squeue -u $USER
- $ scancel <jobid>

- squeue: show running and waiting jobs
- scancel: cancel a job

srun and salloc

- $ salloc --gres=gpu:1 --time=00:30:00
- $ srun ./main

- salloc: reserve resources interactively
- srun: start a program inside an allocation

sinfo and scontrol

- $ sinfo
- $ scontrol show job <jobid>

- sinfo: show partitions and node status
- scontrol: show detailed job, node, and resource information

Summary Slurm

- $ module
- $ sbatch
- $ squeue
- $ scancel
- $ salloc
- $ srun
- $ scontrol
- $ sinfo

Quick CUDA Guide

Example Code

```
#include <iostream>
#include <cuda_runtime.h>

__global__ void vectorAdd(float* A, float* B, float* C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

```
int main() {
    const int N = 1024;
    const int size = N * sizeof(float);
    float h_A[N], h_B[N], h_C[N];
    for (int i = 0; i < N; i++) {
        h_A[i] = i;
        h_B[i] = 2 * i;
    }
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
    std::cout << "Result: " << h_A[5] << " + "
              << h_B[5] << " = " << h_C[5] << std::endl;
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    return 0;
}
```

What Is CUDA?

- CUDA is NVIDIA’s programming model for GPUs
- CPU = host
- GPU = device
- Many threads run in parallel on the GPU

CUDA Program Structure

1. allocate memory
2. copy data Host → Device
3. launch kernel
4. copy data Device → Host
5. free memory

cudaMalloc

- Allocates memory on the GPU
- Pointer refers to device memory

```
int *d_a;
cudaMalloc(&d_a, n * sizeof(int));
```

cudaMemcpy

- Copies data between CPU and GPU
- Copy direction must be specified explicitly

```
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
```

Kernel Launch <<< >>>

- <<< >>> launches a GPU kernel
- First value: number of blocks
- Second value: threads per block

```
add<<<numBlocks, threadsPerBlock>>>(a, b, c);
```

Kernel and __global__

- __global__: function runs on the GPU
- Called from the host
- Many threads execute the same function

```
__global__ void add(int *a, int *b, int *c) {
    int i = threadIdx.x;
    c[i] = a[i] + b[i];
}
```

Thread and Block Indices

- threadIdx: thread index inside a block
- blockIdx: block index inside the grid
- blockDim: number of threads per block
- gridDim: number of blocks in the grid

Global Index

- Unique index for each thread
- Used to access array elements
- One of the most important CUDA patterns

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

Thread Blocks and Global Indices

- Conceptual example:
    - Block 0: local threads 0 1 2 3
    - Block 1: local threads 0 1 2 3
    - Block 2: local threads 0 1 2 3
  - Global data positions:
    - 0 1 2 3 | 4 5 6 7 | 8 9 10 11
- int i = blockIdx.x * blockDim.x + threadIdx.x;

Bounds Check

- Not every thread necessarily maps to valid data
- Prevents invalid memory access
- Should be used for array-based kernels

```
if (i < n) {
    c[i] = a[i] + b[i];
}
```

Threads, Blocks, and Warps

- Thread: smallest execution unit
- Block: group of threads
- Grid: all blocks of a kernel launch
- Warp: group of 32 threads executed together

Minimal CUDA Flow

```
cudaMalloc(...)
cudaMemcpy(..., cudaMemcpyHostToDevice)

kernel<<<blocks, threads>>>(...)

cudaMemcpy(..., cudaMemcpyDeviceToHost)
cudaFree(...)
```

Check for Errors

```
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",               \
                         __FILE__, __LINE__, cudaGetErrorString(err__)); \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)
```

```
CUDA_CHECK(cudaMalloc(&devA, vectorLength*sizeof(float)));
CUDA_CHECK(cudaGetLastError())
CUDA_CHECK(cudaDeviceSynchronize())
```

Measuring runtime

```
#include <chrono>
#include <iostream>

auto t_start = std::chrono::steady_clock::now();
vector_scale<<<blocksPerGrid, threadsPerBlock>>>(d_x, d_y, alpha, n);
cudaDeviceSynchronize();
auto t_stop = std::chrono::steady_clock::now();

double milliseconds = std::chrono::duration<double, std::milli>(t_stop -
t_start).count();

std::cout << "Kernel time: " << milliseconds << " ms\n";
```

Compile CUDA Code

- CUDA source files usually end in .cu
- nvcc compiles host and device code

```
nvcc main.cu -o main
./main
```

Useful CUDA Basics to Add

- cudaFree: free GPU memory
- cudaDeviceSynchronize: wait for GPU work to finish
- dim3: define 1D, 2D, or 3D grid/block layouts
- __shared__: declare shared memory
- __device__: function callable only from GPU code
- __host__: function callable from CPU code

Important Rules of Thumb

- Always use bounds checks
- Explicitly copy data between host and device
- Free GPU memory after use
- Start with correct code, optimize later
- Do not run heavy computations on login nodes

Shared Memory

- Use shared memory for temporary block-local data.
- Synchronize after writing before other threads read.
- Do not use shared memory for communication between blocks.
- Keep shared memory usage small; too much can reduce occupancy.
- Avoid placing __syncthreads() inside branches unless all threads reach it.

```
__shared__ float buffer[BLOCK_SIZE];

int tid = threadIdx.x;

buffer[tid] = local_value;

__syncthreads();
```

Atomic Add

- Use atomicAdd only when several threads may update the same variable.
- Iintialize the target value before the kernel starts.
- Reduce locally first, then use fewer atomicAdd calls.
- Prefer one atomic update per block instead of one per thread when possible.
- Do not use atomicAdd as a global synchronization mechanism.
- Expect floating-point sums to be numerically order-dependent.

```
float contribution = ...;

atomicAdd(result, contribution);
```

Appendix